

Limit Statement

- Limits the number of rows to be displayed when running the code.
- How many rows do you want to see on the result?

Product-ID	Product-name	Price
111	Apple	2
112	Banana	4
113	Orange	3
114	Kiwi	5

Only 2 Rows?

ANS: Select *
FROM Products
LIMIT 2;

Product-ID	Product-name	Price
111	Apple	2
112	Banana	4

SQL Aliases ^{Nicknames} (AS)

- ↳ Form of renaming column names or table name within the SQL script. Temporary name.
- ↳ Used to make column names more descriptive.

eg. Products Table.

```
SELECT product-ID AS barcode,  
       Product-name AS fruitName,  
       Price  
FROM products;
```

barcode	fruitname	Price
111	Apple	2
112	Banana	4
113	Orange	3
114	Kiwi	5

Sorting Statement

- It helps us to arrange our data from either the smallest to biggest, visa versa.

Order By statement → Purpose is to rearrange the data in the table

↳ Allows us to arrange data in **Ascending order (ASC)**

and **Descending Order (DESC)**

↓
Smallest to Biggest
Low to High
New to old
A - Z
1 - 9

↓
Biggest to Smallest
High to Low
Old to new
Z to A
9 to 1

Syntax :
Select (columns)
from database.Schema table
Where condition
Order By column 2 **DESC / ASC**

Table : Products

Product ID	Product name	Rands Price
111	Apple	2
112	Banana	4
113	Orange	3
114	Kiwi	5

Product name + price
order by

Select product-name,
Price

From Products

Order By Price **DESC**

SELECT product-name, Price
FROM Products

Order By Price **ASC**

Product Name	Price
Apple	2
Orange	3
Banana	4
Kiwi	5

Product Name	Price
Kiwi	5
Banana	4
Orange	3
Apple	2

Aggregate functions

- Consolidates data
- Summarises data
- Combines data into groups

→ In SQL aggregate functions summarise many rows into one result / or one result per group.

→ Looks at multiple rows in a table and return a single value.

Types:

- ① **COUNT ()** → Looks at how many rows
→ How many people are in a taxi? 16?
- ② **SUM ()** → Sums / totals data in numeric
- ③ **Average AVG ()** → Gets average of numeric column.
- ④ **Min ()** → Finds the smallest value of a column.
- ⑤ **Max ()** → Finds the largest value in a column.

SPAZA / Products

ID	Fruit-Name	Revenue
101	Apple	2
102	Banana	4
103	Orange	3
104	Kiwi	5

→ How much did you make?

SELECT SUM (Revenue) AS total-revenue
FROM Products;

What is the highest value you made?

SELECT Max (Revenue) AS highest-revenue
FROM Products;

Highest-Revenue
5

What is the smallest revenue?

SELECT min (revenue) AS lowest-rev
FROM products;

Lowest Rev
2

→ **COUNT (DISTINCT ID)**

↳ Used to count the number of different values in a specific column. SELECT COUNT (DISTINCT ID) FROM employees;

How many fruits?

SELECT COUNT (ID) AS number-of-fruits
FROM products;

Number-of-fruits
4

GRADE 10H
M
f
M
f

M=Male
F=female

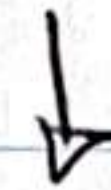
GROUP BY

ID	Name	Gender
101	Mbali	F
102	Rochi	M
103	Thabo	M
104	Jacob	M
105	John	M
106	Lerato	F
107	Robert	M
108	Anna	F

Count number of females & males.

SQL Script:-

SELECT COUNT (ID) AS number-of-students
FROM Students-Grade10H
GROUP BY Gender;



Number-of-students	Gender
5	M
3	F

Sale-ID	Store	Amount
1	Mr Price	100
2	Mr Price	50
3	Pep	80
4	Pep	20

-> You cannot GROUP BY without having a collation column in the SELECT.

-> Do not perform SUM() on an ID or unique identifier

1. Write an SQL query to calculate the total revenue generated by all stores.

SELECT SUM (amount) AS total-revenue
FROM sales;

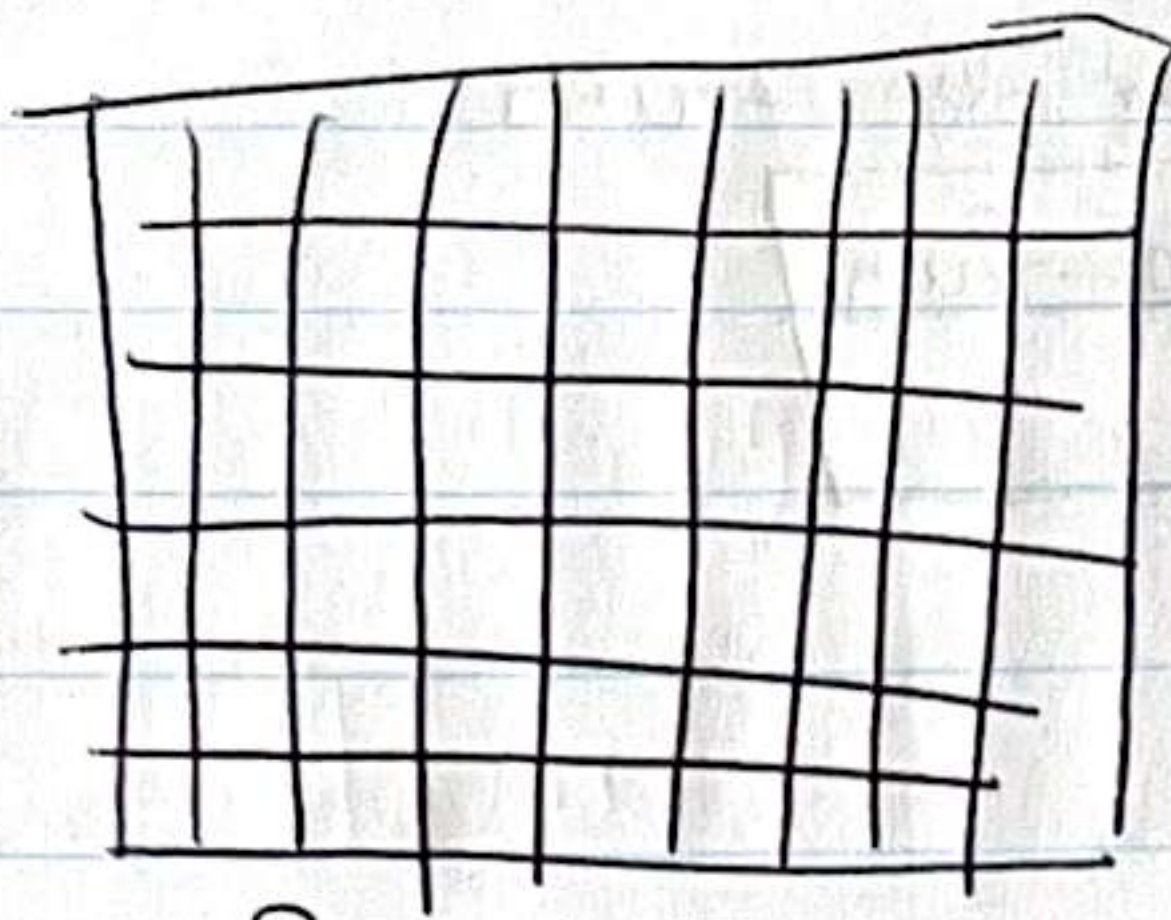
total-revenue
250

2. Write SQL query to calculate the total revenue per store group.

SELECT Store
SUM (amount) AS total revenue
FROM sales
GROUP BY store;

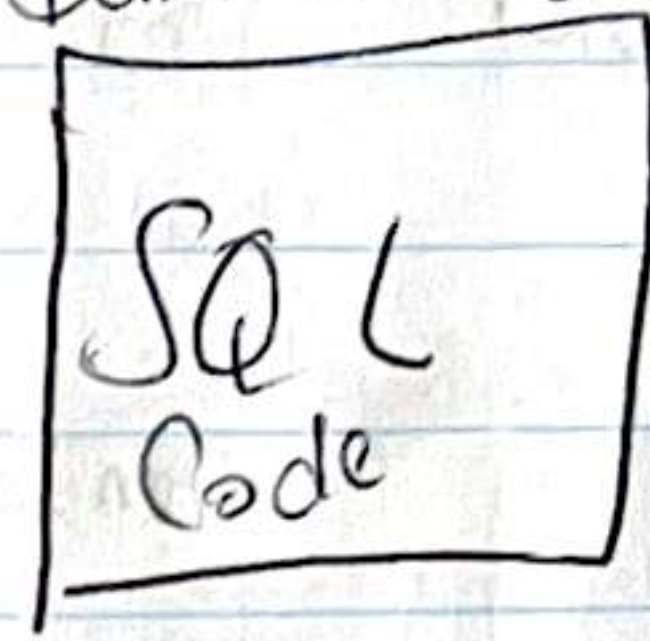
Store	total-revenue
Mr Price	150
Pep	100

SQL



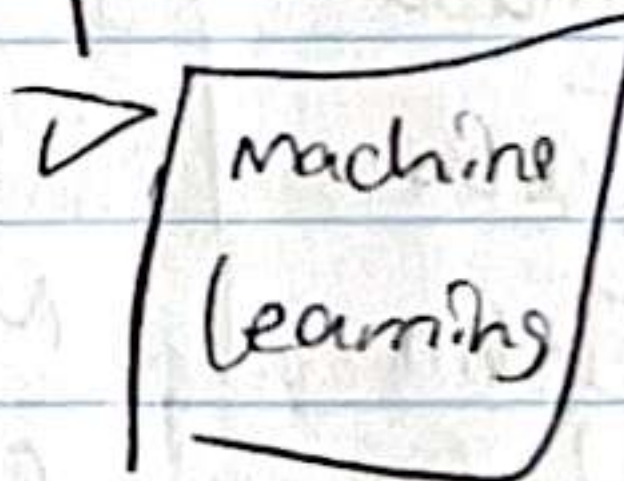
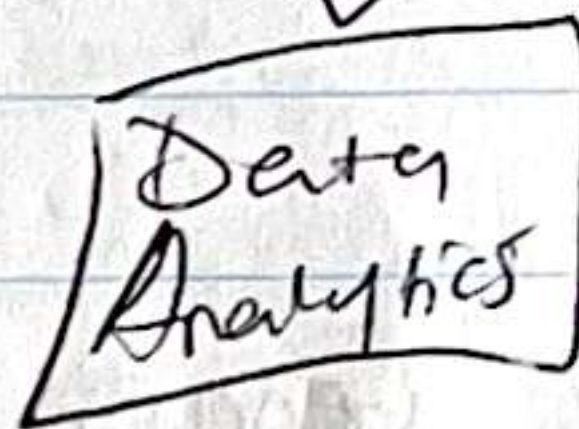
Big data in
warehouse

Data management system



clean up
data

Aggregated data - summarised



Aggregate functions:

- Sum()
- Count()
- min()
- max()
- Avg()

Group By

- Helps to group data into categories e.g male / female.

Having Statements

- Helps us to filter data that is grouped.
- Comes after Group By Statement.

Syntax:

SELECT AGG - function (column) AS new-column-name,
Categorical - column

FROM database.schema.table

GROUP BY categorical-column

HAVING AGG-function (column) condition.

Table: Sales

Sale-ID	Product	Category	Amount
101	Pen	Stationery	10
102	pen	Stationery	15
103	book	Stationery	40
104	mouse	Electronics	200

SELECT Category
S

Calculate which category generates the most revenue?

⇒ SELECT SUM(Amount) AS total-revenue,
Category

FROM sales
GROUP BY

Category	Total Revenue
Stationery	65
Electronics	200

SELECT Category
SUM(Amount)
AS total-revenue
FROM sales
GROUP BY Category

Select max (Amount)
from sales;

Amount	Category
200	Electronics

Select for category that generates R100 +

HAVING - filtering data that is aggregated

ID	Name	Department	Salary
1	Rophiwa	IT	100
2	Mbali	HR	20
3	Lerato	HR	10
4	Sipho	Sales	15
5	Thabo	Sales	10

Calculate total salaries per department

Select department,
Sum (salary) As total-salaries

From Employees

Group By department;

Department	total-salaries
IT	100
HR	30
Sales	25

CASE Syntax

Select column 1,

CASE WHEN condition 1 THEN 'Result 1'
 WHEN condition 2 THEN 'Result 2'

ELSE 'default-result'

END As 'column-name',

column 2,

column 3,

From table